In [61]:
```
import astropy.units as u
import astropy.constants as const
import numpy as np

h  = const.h        # Planck constant
kB = const.k_B      # Boltzmann constant
c  = const.c        # Speed of light
pi = np.pi
G = const.G

np.set_printoptions(precision=6, formatter={'float_kind': '{:.3e}'.format
```

In [62]:
```
d = 8.5 *u.kpc

F = d/(10**57)**2
F.decompose()
```

Out[62]: $2.62283 \times 10^{-94}\ \mathrm{m}$

10.1

In [63]:
```
# a

theta_snr = 1.2 * u.arcmin
V_snr = 14000 * u.km / u.s
d_snr = 8.5 * u.kpc

# small angle aprox: theta = D/d

D_snr = (theta_snr * d_snr) / u.rad
R_snr = .5 * (theta_snr * d_snr) / u.rad
t_0 = R_snr / V_snr
t_0.decompose()


print(f"t_0 = {t_0.to(u.yr).value:.3e} {t_0.to(u.yr).unit}")

D_snr
```

```
t_0 = 1.036e+02 yr
```

Out[63]: $10.2\ \frac{'\,\mathrm{kpc}}{\mathrm{rad}}$

In [64]:
```
# b

e_n = 10 ** (57) * u.def_unit('neutrino')

delta_t = 15 * u.s

# neutrions per m**2 per second
# f = L/(4 * pi * d**2)
F = (e_n/(4 * pi * d_snr**2))
flux = F/delta_t
flux.decompose()


print(f"F = {F.decompose().value:.3e} {F.decompose().unit}")
```

```
print(
'flux =', f"{flux.decompose():.3e}"
)
```

```
F = 1.157e+15 neutrino / m2
flux = 7.712e+13 neutrino / (s m2)
```

In [65]:
```
# c
L_bol = 10**9 * u.L_sun

M_bolS = 4.74
# M_bol = M_bol \odot - 2.5 log(lbol/L_\odot)
M_bol = M_bolS - 2.5*np.log10(L_bol/u.L_sun) # must make it unitless


m_v = M_bol + 5*np.log10(d_snr/ u.pc) - 5
m_v

print(f"M_bol = {M_bol.value:.3e} {M_bol.unit}")

print(f"m_v = {m_v.value:.3e} {m_v.unit}")

-3.113e+00 +30
```

```
M_bol = -1.776e+01
m_v = -3.113e+00
```

Out[65]: 26.887

10.2

In [66]:
```
r_wd = 6000 * u.km
rp_sun = 25.4  * u.day
m_sun = 1 * u.M_sun
r_sun = 1 * u.R_sun

# I = 2/5 M R **2
# omega = theta/t

I_sun = 2/5 * m_sun * r_sun**2
omega_sun = (2 * pi)/rp_sun
L = I_sun * omega_sun

print(f"L = {L.decompose().value:.3e} {L.decompose().unit}")

I_wdSun = 2/5 * m_sun * r_wd**2
omega_wdSun = L/I_wdSun
omega_wdSun.decompose()

P_wdSun = (2 * np.pi / omega_wdSun).to(u.s)
P_wdSun
```

```
L = 1.102e+42 m2 kg / s
```

Out[66]: $163.232\ \mathrm{s}$

10.3

In [67]:
```
# a
M_wd = .7 * u.M_sun
r_wd = 7 * 10**6 *  u.m
```

```
p_wd = 1.337 * u.s

# p**2/a**3 = 4pi**2/(G(m_1+m2))

a = (((p_wd**2)
      /((4 * pi**2)
      /(G * 2 * M_wd)))
     **(1/3) )

a.decompose()

print(f"a = {a.decompose().value:.3e} {a.decompose().unit}")

print(f"a = {a.to(u.AU).value:.3e} {a.to(u.AU).unit}")
```

```
a = 2.034e+06 m
a = 1.360e-05 AU
```

In [68]:
```
# c
## WRONG ANSWER

p_wd = 1.337 * u.s
v_rot = .1 * c * (p_wd/(1 * u.s))**(-1) * (1* u.M_sun/(.7 * u.M_sun))**(-
v_rot.decompose()
print(f"v_rot = {v_rot.decompose().value:.3e} {v_rot.decompose().unit}")
# v_rot = 2 * pi/t

t = (2*np.pi * r_wd / v_rot).to(u.s)

print(f"t = {t.decompose().value:.3e} {t.decompose().unit}")
```

```
v_rot = 1.991e+07 m / s
t = 2.209e+00 s
```

In [69]:
```
# c
#### RIGHT ANSWER
vrot = 2 * pi * .01 * r_sun / u.s

precentof_c = vrot/ c

precentof_c.decompose()
```

Out[69]: 0.145808

In [70]:
```
# e
v_given = .01 * c *(1* u.M_sun/(.7 * u.M_sun))**(1/2)

v = np.sqrt(((G * M_wd)/r_wd))
print(f"v = {v.decompose().value:.3e} {v.decompose().unit}")

print(f"v_given = {v_given.decompose().value:.3e} {v_given.decompose().un
```

```
v = 3.643e+06 m / s
v_given = 3.583e+06 m / s
```

In [71]:
```
# e
m_sun = const.M_sun
r_sun = const.R_sun
val = np.sqrt((G * .7 *m_sun)/( .01 * r_sun))
a = val/c
a.decompose()
```

Out[71]: 0.0121891

In [72]:
```
# g
## WRONG
M = 1.4 * u.Msun
R_ns = 10 * u.km
P = 1.337 * u.s

v_rot  = (2*np.pi * R_ns / P).to(u.m/u.s)
v_crit = np.sqrt((const.G * M) / R_ns).to(u.m/u.s)

print(f"v_rot = {v_rot.decompose().value:.3e} {v_rot.decompose().unit}")
print(f"v_crit = {v_crit.decompose().value:.3e} {v_crit.decompose().unit}
```

```
v_rot = 4.699e+04 m / s
v_crit = 1.363e+08 m / s
```

In [73]:
```
# g

##RIGHT
p = 1.337 * u.s
vrpt = (2*pi*(11*u.km))/p
vrpt.decompose()

v_crit = np.sqrt(G * 1.4 * u.Msun/(11 * u.km))
v_crit.decompose()
```

Out[73]: $1.29964 \times 10^{8} \ \frac{\mathrm{m}}{\mathrm{s}}$

10.4

In [74]:
```
# vera rubin quesiton lmao

v_c0 = 255 * u.km /u.s
r_c0 = .4 * u.kpc

v_c1 = 192 * u.km /u.s
r_c1 = 2.8 * u.kpc

v_c2 = 235 * u.km /u.s
r_c2 = 17 * u.kpc

v_c3 = 220 * u.km /u.s
r_c3 = 8 * u.kpc
# v**2 = GMr

m0 = v_c0**2 * r_c0 / G
m1 = v_c1**2 * r_c1 / G
m2 = v_c2**2 * r_c2 / G
m3 = v_c3**2 * r_c3 / G

print(f"m0 = {m0.decompose().value:.3e} {m0.decompose().unit}")
```

```
print(f"m1 = {m1.decompose().value:.3e} {m1.decompose().unit}")
print(f"m2 = {m2.decompose().value:.3e} {m2.decompose().unit}")
print(f"m3 = {m3.decompose().value:.3e} {m3.decompose().unit}")
```

```
m0 = 1.203e+40 kg
m1 = 4.772e+40 kg
m2 = 4.340e+41 kg
m3 = 1.790e+41 kg
```

In [75]:
```
Vs = [255 * u.km /u.s, 192 * u.km /u.s, 220 * u.km /u.s, 235 * u.km /u.s]
Rs = [ .4 * u.kpc, 2.8 * u.kpc, 8 * u.kpc, 17 * u.kpc]
Ms = []

for i in range(len(Vs)):
    m = Vs[i]**2 * Rs[i] / G
    print(f"m{i} = {m.decompose().value:.3e} {m.decompose().unit}")
    print(f"m{i} = {m.to(u.M_sun).value:.3e} {m.to(u.M_sun).unit}")
```

```
m0 = 1.203e+40 kg
m0 = 6.048e+09 solMass
m1 = 4.772e+40 kg
m1 = 2.400e+10 solMass
m2 = 1.790e+41 kg
m2 = 9.003e+10 solMass
m3 = 4.340e+41 kg
m3 = 2.183e+11 solMass
```

10.5

In [76]:
```
# a
a = 960 * u.AU
p = 15.6 * u.yr
e = .867

# p**2 = (4* pi**2 * a**3)/(G*M)

M_bh = (4* pi**2 * a**3)/(G * p**2)
print(f"M_bh = {M_bh.decompose().value:.3e} {M_bh.decompose().unit}")
```

```
M_bh = 7.229e+36 kg
```

In [77]:
```
# b

r_a = a*(1+e)
r_p = a*(1-e)

print(f"r_a = {r_a.decompose().value:.3e} {r_a.decompose().unit}")
print(f"r_p = {r_p.decompose().value:.3e} {r_p.decompose().unit}")


v_a = (((G*M_bh)/a) * ((1-e)/(1+e)))**(1/2)
v_pe = (((G*M_bh)/a)* ((1+e)/(1-e)))**(1/2)

v_ac = v_a / c *100
v_pec = v_pe /c * 100

print(f"v_a = {v_a.decompose().value:.3e} {v_a.decompose().unit}")

print(f"v_pe = {v_pe.decompose().value:.3e} {v_pe.decompose().unit}")

print(f"v_ac = {v_ac.decompose().value:.3e} {v_ac.decompose().unit}")
```

```
print(f"v_pec = {v_pec.decompose().value:.3e} {v_pec.decompose().unit}")

print(f"v_ac = {v_ac.decompose()} {v_ac.decompose().unit}")
print(f"v_pec = {v_pec.decompose()} {v_pec.decompose().unit}")

print(f"r_p = {r_p.to(u.AU).value:.3e} {r_p.to(u.AU).unit}")
```

```
r_a = 2.681e+14 m
r_p = 1.910e+13 m
v_a = 4.892e+05 m / s
v_pe = 6.867e+06 m / s
v_ac = 1.632e-01
v_pec = 2.291e+00
v_ac = 0.1631852520416201
v_pec = 2.290728312494021
r_p = 1.277e+02 AU
```

In [78]:
```
# c
m = 17.5 * u.M_sun
r = 7.4 * u.R_sun

# d = R_m*(2 *(M)/m)**(1/3)

d = r*((2*M_bh)/m)**(1/3)

closest = r_p/d
print(
'RL =', d.decompose()
)

print(
'closest =', closest.decompose(),
    'times the distance of RL'
)

print(f"d = {d.decompose().value:.3e} {d.decompose().unit}")
```

```
RL = 384159290930.25507 m
closest = 49.72066687421016 times the distance of RL
d = 3.842e+11 m
```